UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



Programación Concurrente y Paralela
Trabajo Práctico N°1

Docentes: Ing. Luis Orlando Ventre, Ing. Mauricio Ludemann y Aspirante Docente Ing. Agustin Carranza

Año: 2026
Semestre: Segundo Semestre

Apellido y Nombre: *Bastía, Álvaro*
DNI: *44.783.012*
Apellido y Nombre: *Britez, Carlos*
DNI: *44.221.771*
Apellido y Nombre: *Género, Sol*
DNI: *46.541.859*
Apellido y Nombre: *Tarrago, Emilio*
DNI: *44.198.050*
Apellido y Nombre: *Usandivaras, Hebe Azul*
DNI: *44.768.075*

Índice:

Introducción: 3
Consigna: 3
Procedimiento: 3
Diagrama de clases: 3
Diagrama de secuencia: 5
Código: 7
PrinterFarm: 7
OrderSource: 8
ValidationWorker: 9
Resultados: 13
elementos.csv: 13
eventos.csv: 14
resumen.properties: 14
Conclusión: 15

Introducción:

En el presente documento se encuentra registrada la información referente al Trabajo Práctico N° 1 de Programación Concurrente y Paralela. El mismo engloba los primeros temas enseñados en la primera parte de la materia.

Consigna:

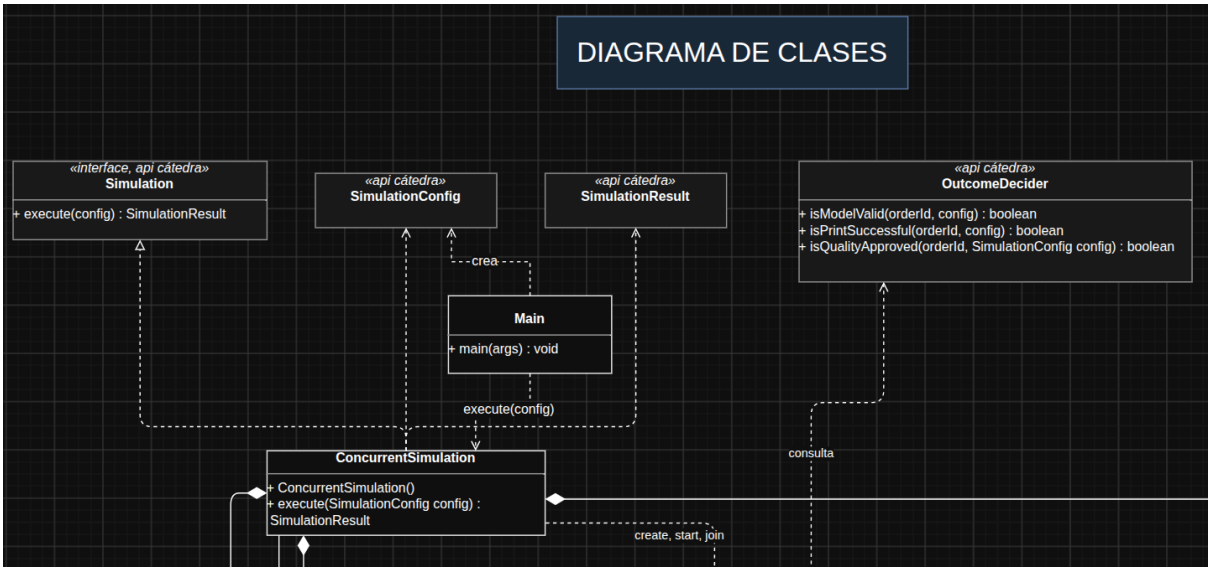
Desarrollar un sistema concurrente que simule una granja industrial de impresión 3D. El sistema recibe órdenes de fabricación, les asigna una impresora, valida los modelos, ejecuta las impresiones y controla la calidad de las piezas obtenidas.

Todas las etapas deben funcionar concurrentemente y compartir recursos de manera segura. El sistema debe procesar todas las órdenes, conservar la consistencia de sus estados y finalizar sin dejar hilos activos.

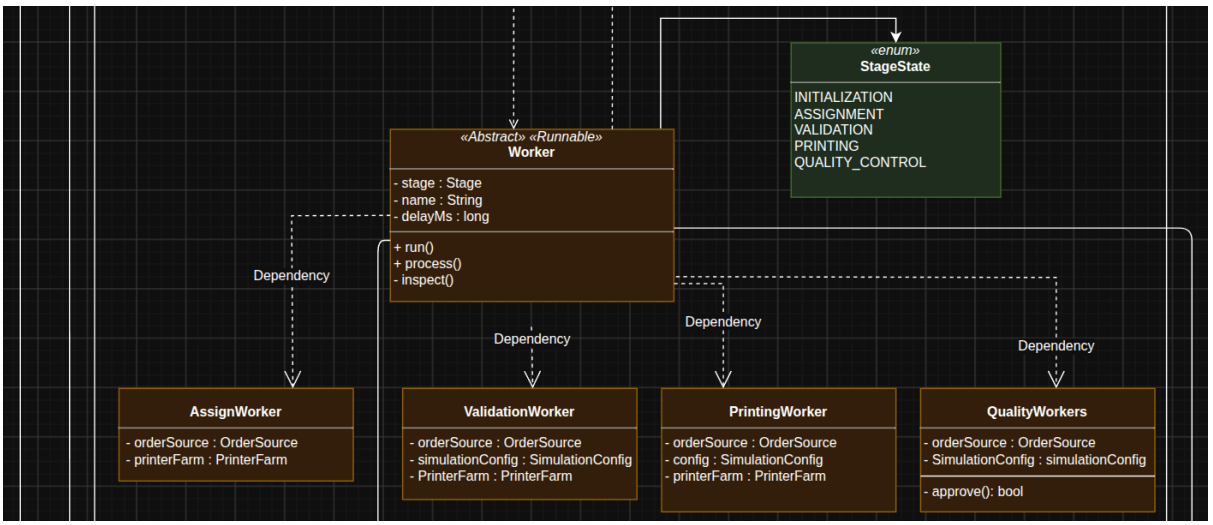
Procedimiento:

Empezamos resolviendo la consigna planteando el diseño de las clases que vamos a requerir, y el uso/recorrido de las mismas con un diagrama de clases y un diagrama de secuencia.

Diagrama de clases:

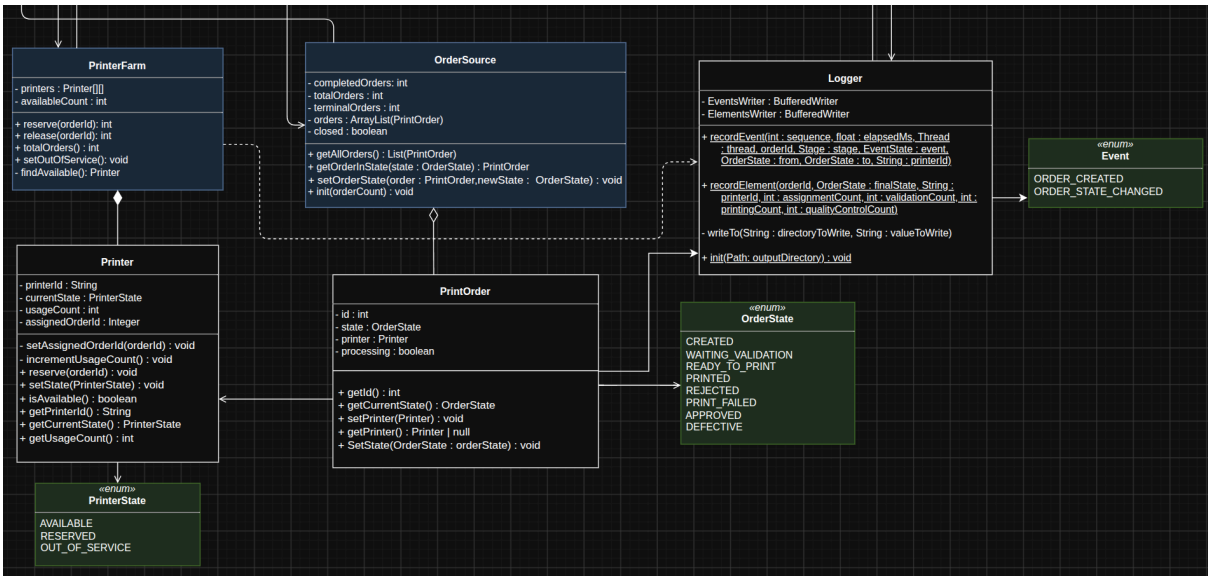


Esta parte del diagrama representa las clases dadas por la cátedra siendo relacionada por la clase ConcurrentSimulation donde aparece el método execute().



Aquí empiezan a aparecer las clases programadas y diseñadas por nosotros, las clases de color azul representan las clases que abarcan las Secciones Críticas y la concurrencia.

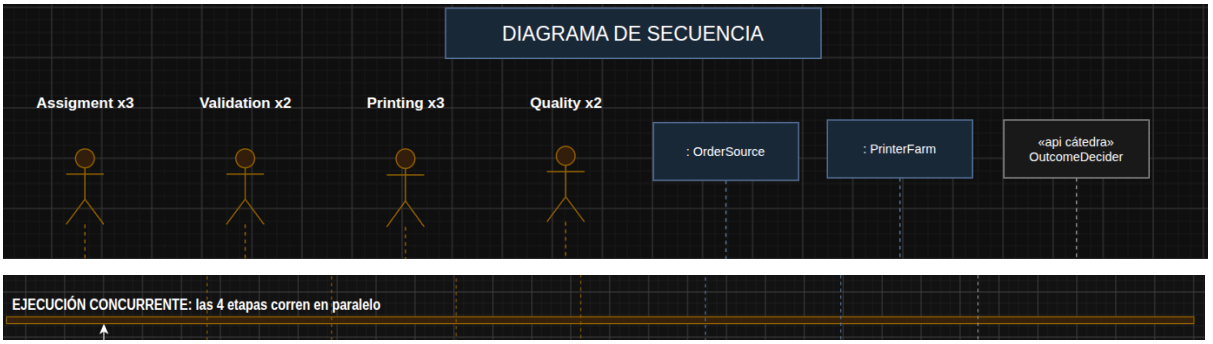
Luego los recuadros de color verde representan ENUMS y las clases naranjas, menos la clase Worker que es una clase abstracta, de la cual el resto de clases naranjas van a extenderse, éstos son los mismos hilos que van a realizar sus respectivas tareas.

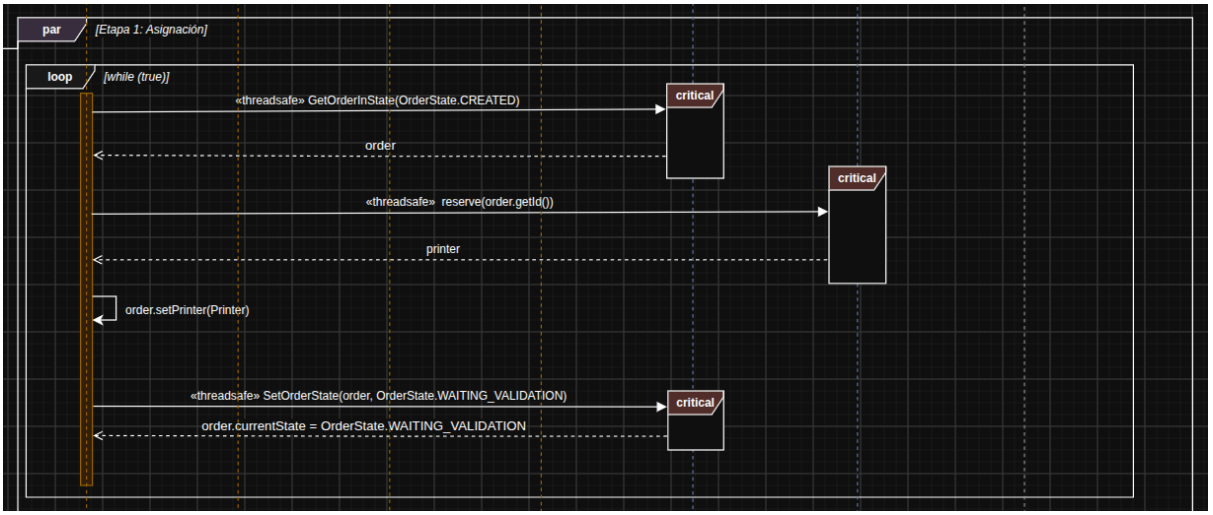


Y por último las clases de color gris representan las clases donde estan los datos y recursos.

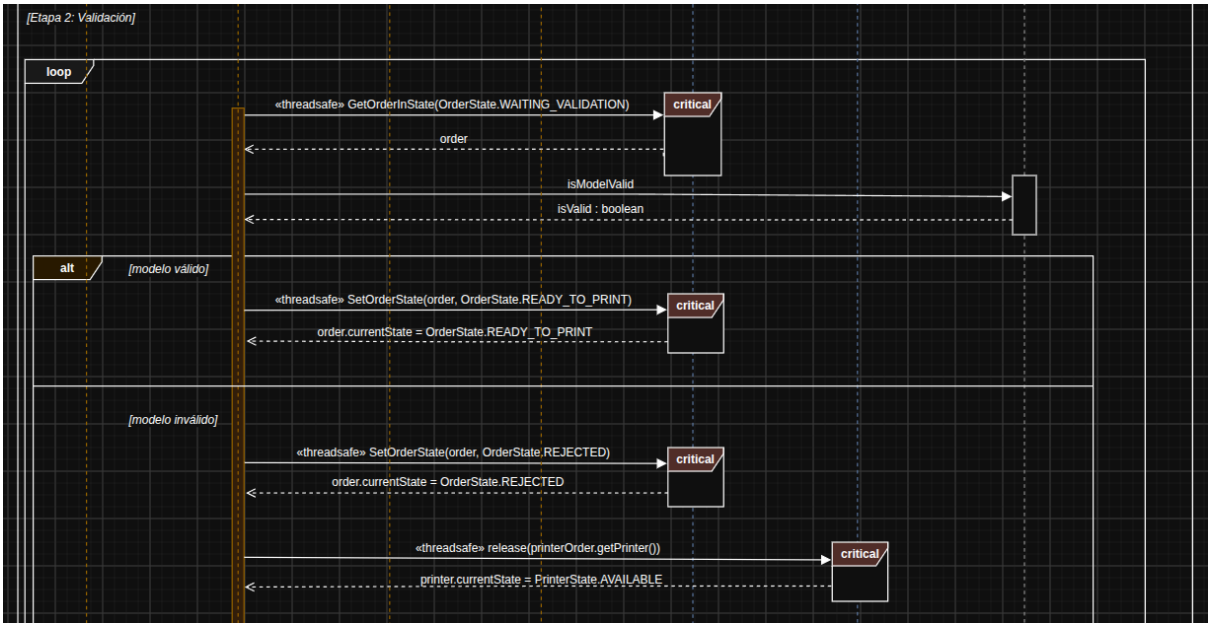
Diagrama de secuencia:

Decidimos diseñar la parte más importante de todo el programa, la concurrencia. Este diagrama implementa el recorrido de las 4 etapas (Los Workers) ocurriendo en paralelo y, al mismo tiempo, más de un hilo encargándose de esa etapa, por lo tanto la concurrencia.

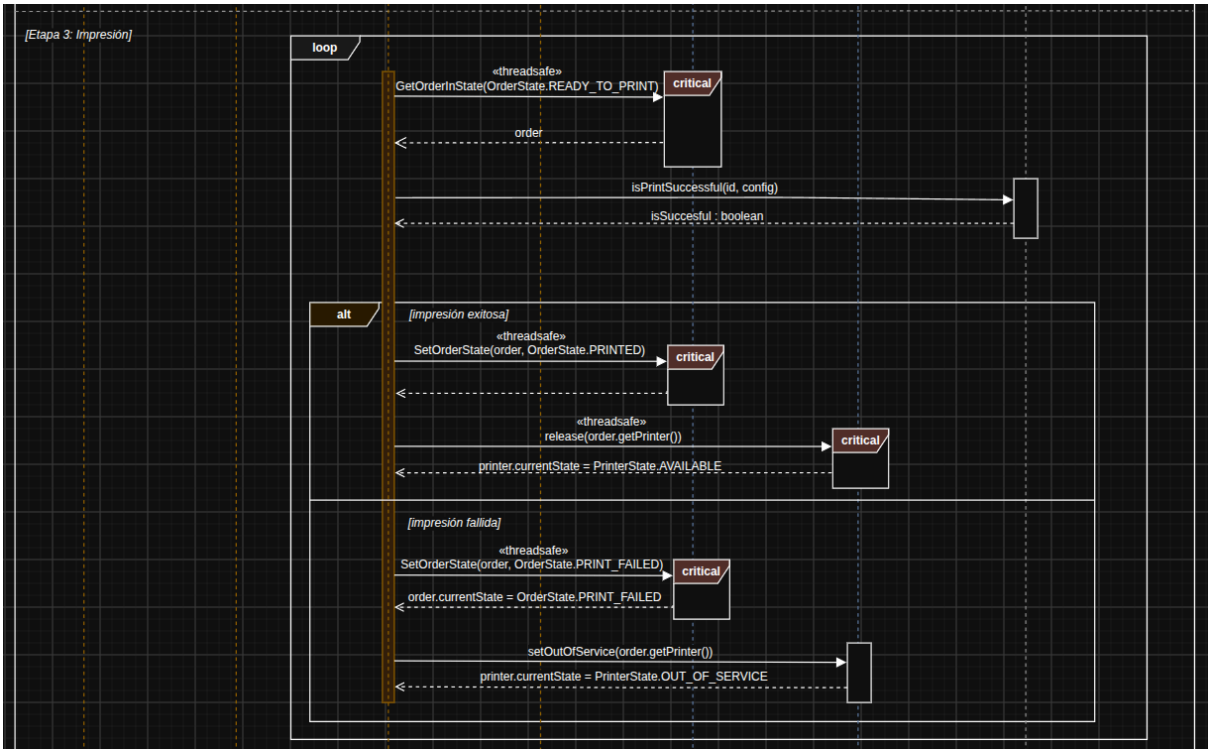




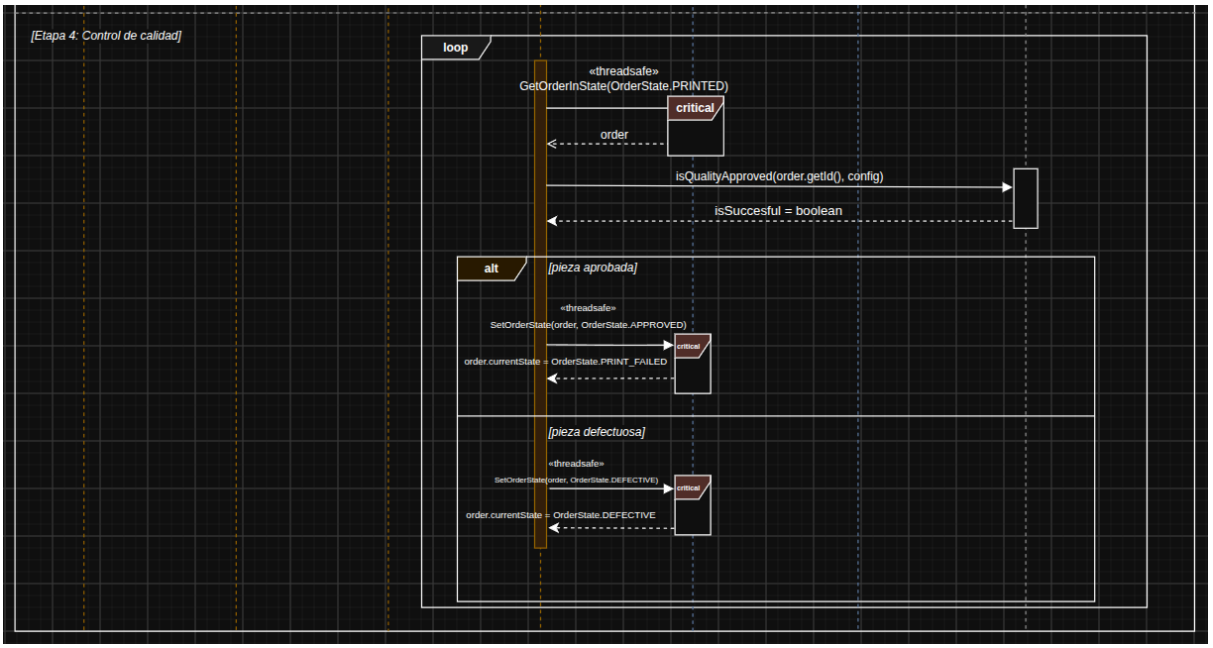
Este es el recorrido que realiza la Etapa 1 - Asignación.



A partir de la etapa 2, el recorrido empieza a recurrir a la clase OutcomeDecider dada por la cátedra.



Apaciblemente, con cada etapa aumentan la cantidad de métodos, instrucciones que se realizan, mientras que el manejo de la concurrencia en las secciones críticas se mantiene relativamente similar.



Por último la Etapa 4 - Control de Calidad. Aunque no se llegue a apreciar del todo en el diagrama, estas 4 etapas están ocurriendo al mismo tiempo, constantemente esperando (recursos disponibles) o realizando su correspondiente tarea, mientras que alteran los recursos que todas las etapas están usando y que cada hilo de esas etapas.

Código:

A partir de estos diagramas planificamos el código para respetar y conseguir el flujo de las etapas con concurrencia entre los hilos. Los métodos más importantes los cuales debían ser protegidos con una sección crítica, fueron los siguientes.

PrinterFarm:

La clase se encarga de las secciones críticas referidas al manejo y uso de las impresoras. Sus métodos más importantes son:

```
public synchronized Printer reserve(PrintOrder order) throws InterruptedException {  19 usages
    while (availableCount == 0) {
        wait(); // soltamos el lock y duerme; al despertar vuelve a chequear la condicion
    }
    Printer printer = findAvailable();
    printer.reserve(order.getId());   // AVAILABLE -> RESERVED, suma usos, guarda la orden
    availableCount--;
    return printer;
}
```

```
public synchronized void release(Printer printer) {
    printer.setState(PrinterState.AVAILABLE);
    availableCount++;
    notifyAll();
}
```

```
public synchronized void setOutOfService(Printer printer) {
    printer.setState(PrinterState.OUT_OF_SERVICE);
}
```

Estos métodos se encargan mediante la keyword **synchronized** a manejar los estados de las impresoras y sus respectivos contadores. Además se llama el **notifyAll()** y **wait()** para los casos donde no se encuentran impresoras disponibles, una vez se libera una impresora se manda una notificación para que un hilo pueda buscar una impresora disponible.

OrderSource:

Clase que también maneja secciones críticas, en esta clase se introduce un solo **key object** llamado **LOCK** que permite llamar un hilo a la vez entre **getOrderInState()** y en **setOrderState()** para que no se pueda cambiar de estado al mismo tiempo que otro hilo estaba buscando ese estado. Aquí también se utiliza el **wait()** y **notifyAll()** por razones similares a la clase anteriormente mencionada.

```
public PrintOrder getOrderInState(OrderState wanted) throws InterruptedException {  4 usag
    synchronized (LOCK) {
        while (true) {
            for (PrintOrder o : orders) {
                if (o.getCurrentState() == wanted && !o.processing) {

                    switch (wanted) {
                        case OrderState.CREATED: o.assignmentCount ++; break;
                        case OrderState.WAITING_VALIDATION: o.validationCount ++; break;
                        case OrderState.READY_TO_PRINT: o.printingCount ++; break;
                        case OrderState.PRINTED: o.qualityControlCount ++; break;
                        default: break;
                    }

                    o.processing = true;
                    return o;
                }
            }
            if (terminalCount == orders.size()) return null;   // el worker termina
            LOCK.wait();
        }
    }
}
```

```
public void SetOrderState(PrintOrder order,OrderState newState, Worker worker, Runnable printerAction) {
    synchronized (LOCK) {
        if (printerAction != null) printerAction.run();
        OrderState prev = order.getCurrentState();
        order.SetState(newState);
        order.processing = false;

        // Se registra DENTRO del lock: las filas de eventos.csv quedan en el mismo orden
        // en que se confirman las transiciones. La secuencia la numera el Logger.
        Logger.recordEvent(
            worker,
            order.getId(),
            prev, newState,
            order.getPrinter() == null ? null : order.getPrinter().getPrinterId()
        );

        if (newState.isTerminal()) {
            terminalCount++;
        }

        //  EVENTO CAMBIO DE ESTADO
        LOCK.notifyAll();
    }
}
```

AssignWorker:

La Clase encargada de la ETAPA 1, ésta misa se encarga de todo lo que pide esta etapa y será instanciada por 3 hilos, su único método:

```
@Override 1 usage  E1000io-bit +2
protected void process() throws InterruptedException{
    //Tomo una orden en estado CREATED (bloqueante: si no hay, duermo en el monitor de OrderSource)
    PrintOrder printOrder = orderSource.getOrderInState(OrderState.CREATED);

    //null = todas las ordenes ya son terminales: no queda trabajo, el worker termina
    if (printOrder == null) {
        Thread.currentThread().interrupt();
        return;
    }

    //Obtengo una impresora disponible y la reservo de manera exclusiva
    //(dentro del monitor de PrinterFarm: AVAILABLE -> RESERVED y suma un uso)
    //Si no hay ninguna disponible, el hilo duerme en PrinterFarm hasta que alguien haga release
    Printer printer = printerFarm.reserve(printOrder);

    //La asocio con la orden ANTES de publicar el cambio de estado: cuando otro hilo vea la orden
    //en WAITING_VALIDATION (a traves del lock de OrderSource) ya va a tener su impresora (invariante 5)
    printOrder.setPrinter(printer);

    //Cambio la orden a WAITING_VALIDATION
    orderSource.SetOrderState(printOrder, OrderState.WAITING_VALIDATION, worker: this);
}
```

Este método sobrescribe el método que proviene de su Clase padre llamada Worker.

ValidationWorker:

ETAPA 2 con el mismo método sobrescrito que la clase anterior.

```
@Override 1 usage  E1000io-bit +3
protected void process() throws InterruptedException {
    //Tomo una orden en WAITING_VALIDATION
    PrintOrder printOrder = orderSource.getOrderInState(OrderState.WAITING_VALIDATION);

    //null = todas las ordenes ya son terminales: no queda trabajo, el worker termina
    if (printOrder == null) {
        Thread.currentThread().interrupt();
        return;
    }

    //Decision determinista (fuera de cualquier lock: no toca estado compartido)
    if(OutcomeDecider.isModelValid(printOrder.getId(), simulationConfig)){
        //Si el modelo es valido:
        //Cambia la orden de WAITING_VALIDATION -> READY_TO_PRINT, la impresora permanece reservada
        orderSource.SetOrderState(printOrder,OrderState.READY_TO_PRINT, worker: this);
    }else{
        //Si el modelo no es valido:
        //1) Libero la impresora DE ESTA ORDEN a traves de PrinterFarm (RESERVED -> AVAILABLE).
        //   Va por el monitor de la granja para que availableCount++ y notifyAll() despierten
        //   a los AssignWorker que esperan una impresora.
        //   Se libera ANTES de pasar a REJECTED para que nunca exista una orden terminal con
        //   impresora reservada (invariante 6).
        //2) Cambia la orden de WAITING_VALIDATION -> REJECTED, la orden queda terminada.
        //   La orden conserva la referencia a la impresora como dato historico (elementos.csv).
        //   1) y 2) se hacen juntos dentro del lock de OrderSource (atomico)
        orderSource.SetOrderState(printOrder,OrderState.REJECTED, worker: this,
                () -> printerFarm.release(printOrder.getPrinter()));
    }
}
```

PrintingWorker:

ETAPA 3, y su método process sobrescrito.

```java
@Override 1 usage  E1000io-bit +1 *
protected void process() throws InterruptedException {
    PrintOrder readyToPrintOrder = orderSource.getOrderInState(OrderState.READY_TO_PRINT); //  gets a READY_TO_PRINT order
                    // printer's order from previous stage
    // null = todas las ordenes ya son terminales: no queda trabajo, el worker termina
    if (readyToPrintOrder == null) {
        Thread.currentThread().interrupt();
        return;
    }
    // (la demora la aplica Worker.run() una vez por orden: aca no se duerme otra vez)

    if (OutcomeDecider.isPrintSuccessful(readyToPrintOrder.getId(), this.config)) {

        orderSource.SetOrderState(readyToPrintOrder, OrderState.PRINTED, worker: this,   // READY_TO_PRINT -> PRINTED
                () -> printerFarm.release(readyToPrintOrder.getPrinter()));      // RESERVED -> AVAILABLE (atomico)

    } else {

        Printer printer = readyToPrintOrder.getPrinter();
        orderSource.SetOrderState(readyToPrintOrder, OrderState.PRINT_FAILED, worker: this,  // READY_TO_PRINT -> PRINT_FAILED
                                ()->printerFarm.setOutOfService(printer));          // RESERVED -> OUT_OF_SERVICE (atomico)
    }
}
```

QualityWorker:

Con esta clase terminamos las que se encargan de las etapas, realizando la ETAPA 4 con su sobrescritura del método process:

```java
@Override 1 usage  E1000io-bit +2
protected void process() throws InterruptedException {

    //Tomo una orden PRINTED (bloqueante: si no hay, el hilo duerme en el monitor de OrderSource)
    PrintOrder printedOrder = orderSource.getOrderInState(OrderState.PRINTED);

    // si es null entonces todas las ordenes ya son terminales: no queda trabajo para esta etapa
    if (printedOrder == null) {
        // Marcamos fin del worker: el while de Worker.run() corta en la proxima vuelta
        // (y el Thread.sleep posterior lanza InterruptedException enseguida, sin demorar).
        Thread.currentThread().interrupt();
        return;
    }

    //Transicion de estado DENTRO de OrderSource (lock + notifyAll + conteo de terminales)
    if (isApproved(printedOrder)) {
        orderSource.SetOrderState(printedOrder, OrderState.APPROVED,  worker: this);  // PRINTED -> APPROVED
    } else {
        orderSource.SetOrderState(printedOrder, OrderState.DEFECTIVE,  worker: this); // PRINTED -> DEFECTIVE
    }

    // La demora (qualityControlDelayMillis) la aplica Worker.run() una vez por orden procesada.
    // No hay que liberar impresora: ya se libero en la etapa de impresion.
}
```

Tests:

Implementamos los tests para comprobar que la concurrencia ocurre de forma correcta y no se corrompen o modifican los datos al terminar el programa. A continuación mostramos los tests y sus resultados:

```java
@Test  E1000io-bit
@DisplayName("reserve: la impresora queda RESERVED, asociada a la orden y con un uso")
void reserveAsociaLaOrden() throws InterruptedException {
    PrinterFarm farm = new PrinterFarm( rows: 2,  columns: 2);

    Printer printer = farm.reserve(new PrintOrder( id: 7));

    assertEquals(PrinterState.RESERVED, printer.getCurrentState());
    assertEquals( expected: 7, printer.getAssignedOrderId());
    assertEquals( expected: 1, printer.getUsageCount());
    assertEquals( expected: 1, farm.totalOrders());
}
```

```java
@Test  E1000io-bit
@DisplayName("release: la impresora vuelve a AVAILABLE, sin orden, y se puede volver a reservar")
void releaseLaDejaReutilizable() throws InterruptedException {
    PrinterFarm farm = new PrinterFarm( rows: 1,  columns: 1);
    Printer printer = farm.reserve(new PrintOrder( id: 1));

    farm.release(printer);

    assertEquals(PrinterState.AVAILABLE, printer.getCurrentState());
    assertNull(printer.getAssignedOrderId());

    Printer again = farm.reserve(new PrintOrder( id: 2));
    assertSame(printer, again);
    assertEquals( expected: 2, again.getAssignedOrderId());
    assertEquals( expected: 2, again.getUsageCount());
}
```

```java
@Test  Hebe
@DisplayName("reserve: pasa a RESERVED, suma un uso y guarda la orden")
void reserveMarcaReservadaYGuardaOrden() {
    Printer printer = new Printer( row: 0,  col: 0);

    printer.reserve( orderId: 42);

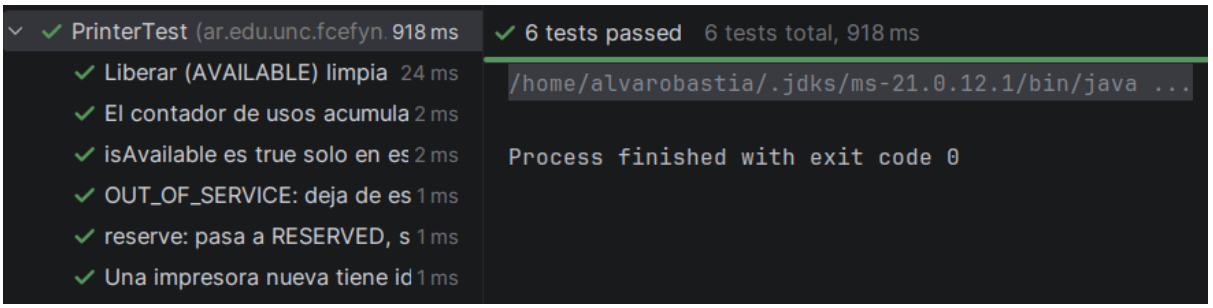
    assertEquals(PrinterState.RESERVED, printer.getCurrentState());
    assertFalse(printer.isAvailable());
    assertEquals( expected: 1, printer.getUsageCount());
    assertEquals( expected: 42, printer.getAssignedOrderId());
}
```

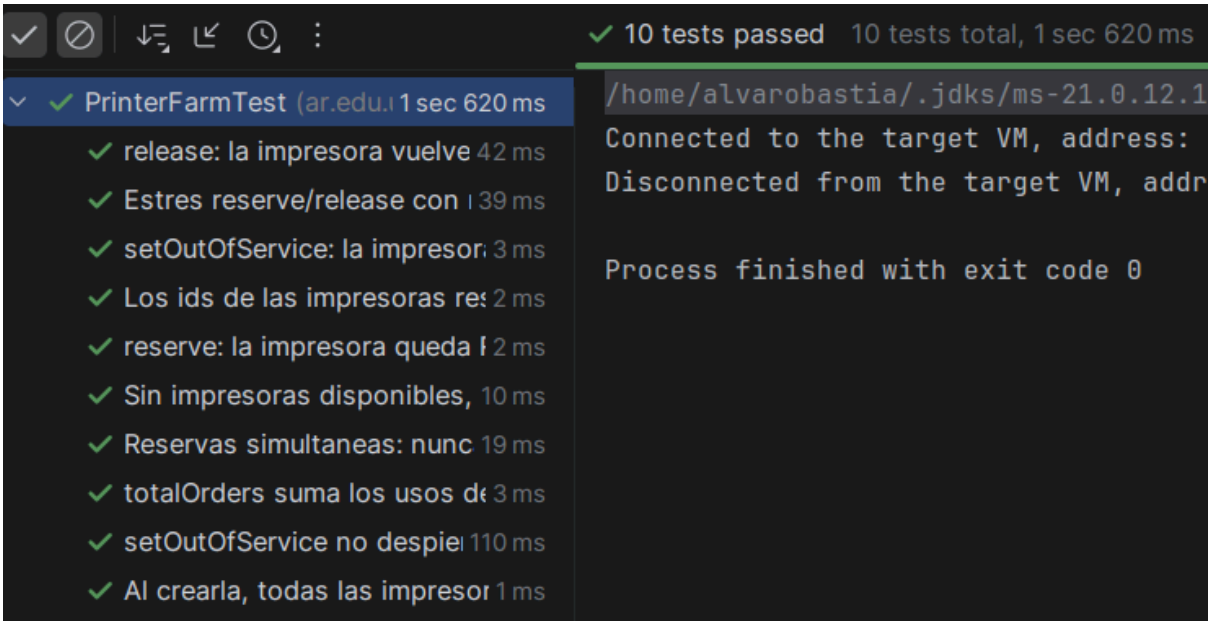
```java
@Test  Hebe
@DisplayName("Liberar (AVAILABLE) limpia la orden asignada pero conserva el contador de usos")
void liberarLimpiaOrdenYConservaUsos() {
    Printer printer = new Printer( row: 0,  col: 0);
    printer.reserve( orderId: 1);

    printer.setState(PrinterState.AVAILABLE);

    assertTrue(printer.isAvailable());
    assertNull(printer.getAssignedOrderId());
    assertEquals( expected: 1, printer.getUsageCount());
}
```

Estos 4 tests son de la clase PrinterFarm y Printer, a continuación los resultados de todos los tests ejecutados:



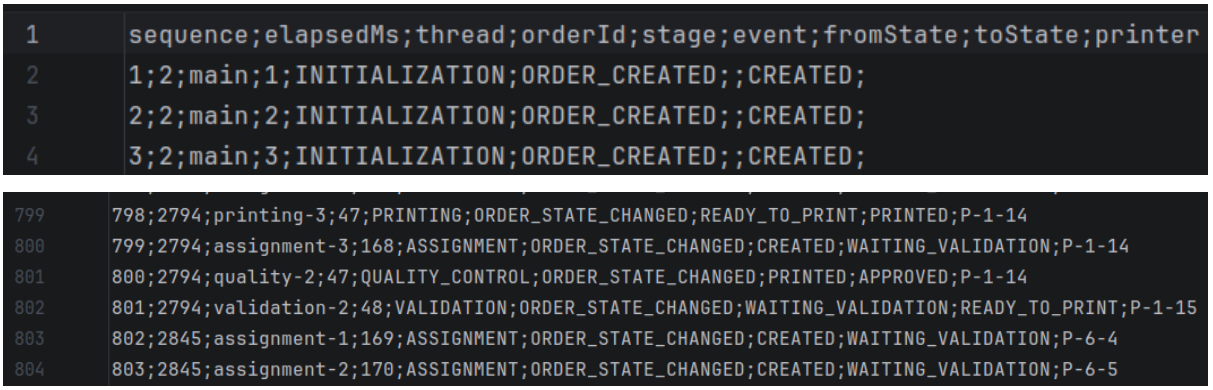


Resultados:

Luego de que el programa principal se termine de ejecutar, se guardan en la carpeta resultados, 3 archivos importantes:

elementos.csv:

Este mismo escribe en un archivo de texto lo que ocurre dentro de cada cambio de estado de las órdenes, a continuación una imagen con el archivo escrito.



```
1    sequence;elapsedMs;thread;orderId;stage;event;fromState;toState;printer
2    1;2;main;1;INITIALIZATION;ORDER_CREATED;;CREATED;
3    2;2;main;2;INITIALIZATION;ORDER_CREATED;;CREATED;
4    3;2;main;3;INITIALIZATION;ORDER_CREATED;;CREATED;
```

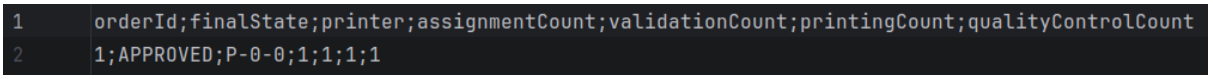


```
799  798;2794;printing-3;47;PRINTING;ORDER_STATE_CHANGED;READY_TO_PRINT;PRINTED;P-1-14
800  799;2794;assignment-3;168;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-1-14
801  800;2794;quality-2;47;QUALITY_CONTROL;ORDER_STATE_CHANGED;PRINTED;APPROVED;P-1-14
802  801;2794;validation-2;48;VALIDATION;ORDER_STATE_CHANGED;WAITING_VALIDATION;READY_TO_PRINT;P-1-15
803  802;2845;assignment-1;169;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-6-4
804  803;2845;assignment-2;170;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-6-5
```

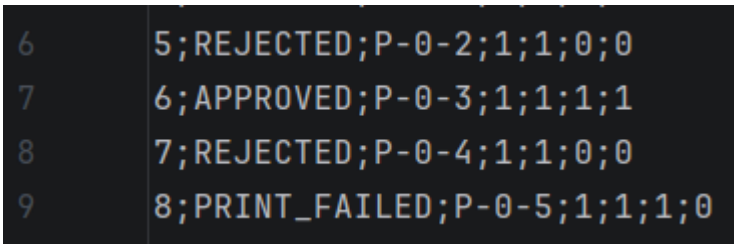
Se pueden observar las mismas etapas y estados por las que pasan las distintas órdenes con sus respectivas impresoras. La línea número 1 representa el header con la información que está entre cada ‘;’.

eventos.csv:

Este archivo deja asentado lo que ocurre con todas las órdenes al final de la ejecución, a continuación una imagen con el archivo escrito.



```
1    orderId;finalState;printer;assignmentCount;validationCount;printingCount;qualityControlCount
2    1;APPROVED;P-0-0;1;1;1;1
```

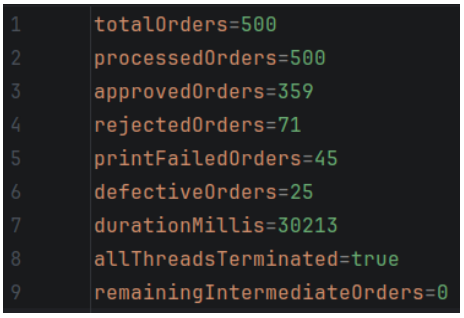


```
6    5;REJECTED;P-0-2;1;1;0;0
7    6;APPROVED;P-0-3;1;1;1;1
8    7;REJECTED;P-0-4;1;1;0;0
9    8;PRINT_FAILED;P-0-5;1;1;1;0
```

El mismo muestra el estado final que quedó cada orden, su impresora asociada durante el proceso (si es que llegó a asociarse a una) y los valores totales de asignación, validación, printeo y control de calidad.

resumen.properties:

Este archivo muestra un mini pantallazo general en números de los datos más importantes.



Una estadística única por cada programa corrido, este puede variar gracias a los condicionales dados por la cátedra en OutcomeDecider.

Conclusión:

Primer trabajo práctico de la materia donde se adentra de lleno a la concurrencia de múltiples hilos con múltiples etapas paralelas. Logramos aprender cómo diseñar la concurrencia en un diagrama de flujos, entender que clases realizan secciones críticas a partir de los métodos señalados en el diagrama de clases, generamos distintas formas de manipular la concurrencia con los bloques synchronized, aprendimos a cómo explicar la elección de la forma elegida para batallar estos problemas y todo esto mientras se trabajaba en equipo a través de un repositorio en github.

Un trabajo que definitivamente se puede mejorar, expandir y realizar de otras formas, pero nos quedamos contentos con el resultado y las decisiones tomadas.